

HW10

Sam Ly

April 17, 2026

Chapter 20

<https://github.com/samlyme/Assignments/tree/cs4080/ch20/main/CS4080>

1. Add support for keys of the other primitive types: numbers, Booleans, and nil. Later, clox will support user-defined classes. If we want to support keys that are instances of those classes, what kind of complexity does that add?

First, we create a general ObjHashable, so we no longer hardcode in ObjString as our default hash key. Then, we use the struct inheritance trick covered earlier.

```
struct Obj {
    ObjType type;
    struct Obj* next;
};

struct ObjHashable {
    Obj obj;
    void* data;
    size_t length;
    uint32_t hash;
};

struct ObjString {
    ObjHashable objHashable;
    int length;
    char* chars;
};
```

The rest is just fixing type casts or struct member names.

2. Look up a few hash table implementations in different open source systems, research the choices they made, and try to figure out why they did things that way.

Rust's standard library has a hashmap implementation that uses the Swisstable algorithm. First, there is always a power of 2 capacity. This is because modulo a power of 2 can be optimized away as a bit manipulation by compilers. Then, we use the similar EMPTY, DELTED(tombstone), and FULL states, but we explicitly encode them as a "control byte" in the entry type, rather than how we do it, with a null pointer. This is just a choice they made.

For the actual algorithm, the key difference is that they implement SIMD byte matching. That means large chunks of the open-addressed table are searched in parallel. This means that, most of the time, only one "walk" is needed to find your key. We can also do this for ours, but requires some extra work.

Here is the key implementation difference: probing is done with larger steps. Specifically, each "probe", we jump by 1 chunk, then 2, then 3. This leaves larger gaps in our table, which is good, since it increases uniformity of the distribution of hashed values. Also, this is still guaranteed to terminate if there is an empty bucket for some mathematical reason.

This is the main design choice difference, and it is pretty interesting. There are some other minute details and edge cases to take care of, but this is the crux of the implementation.

3. Write a handful of different benchmark programs to validate our hash table implementation. How does the performance vary between them? Why did you choose the specific test cases you chose?

```
void benchmarkTable(const char *filePath) {
    FILE* file = fopen(filePath, "r");
    if (file == NULL) {
        fprintf(stderr, "Could not open file '%s'.\n", filePath);
        return;
    }

    // each line is <key>:<value>, assume is just string for now.
    Table table;
    initTable(&table);

    char line[1024];
    char key[128];
    char value[128];
    size_t totalLines = 0;
    size_t validEntries = 0;
    size_t invalidLines = 0;
    size_t bytesRead = 0;
    clock_t start = clock();

    while (fgets(line, sizeof(line), file)) {
        totalLines++;
        bytesRead += strlen(line);
        line[strcspn(line, "\n")] = '\0';

        if (sscanf(line, "%127[^:]:%127s", key, value) == 2) {
            ObjString* k = copyString(key, (int)strlen(key));
            Value v = OBJ_VAL(copyString(value, (int)strlen(value)));

            tableSet(&table, k, v);
            validEntries++;
        } else {
            fprintf(stderr, "Invalid line: %s\n", line);
            invalidLines++;
        }
    }

    clock_t end = clock();
    double elapsedSeconds = (double)(end - start) / CLOCKS_PER_SEC;
    double entriesPerSecond =
        elapsedSeconds > 0.0 ? validEntries / elapsedSeconds : 0.0;
    double megabytesPerSecond =
        elapsedSeconds > 0.0 ? (bytesRead / (1024.0 * 1024.0)) / elapsedSeconds
        : 0.0;
}
```

```

fclose(file);

printf("Benchmark: %s\n", filePath);
printf("  lines read: %zu\n", totalLines);
printf("  valid entries: %zu\n", validEntries);
printf("  invalid lines: %zu\n", invalidLines);
printf("  bytes read: %zu\n", bytesRead);
printf("  table count: %d\n", table.count);
printf("  table capacity: %d\n", table.capacity);
printf("  elapsed: %.6f s\n", elapsedSeconds);
printf("  throughput: %.2f entries/s, %.2f MiB/s\n",
       entriesPerSecond, megabytesPerSecond);

freeTable(&table);
}

```

Overall, pretty good.